Velocity Command Center

Complete Codebase & Auto-Unpack Bundle (v2.1)

Dette dokument indeholder både en **enkeltstående Termux/Linux Auto-Unpack script-blok**, der udpakker hele mappestrukturen og alle kildekodefiler med ét enkelt kommando-kald, samt samtlige filer i fuld længde til direkte gennemgang.

1\. Hurtig Udpakning i Termux / Linux (unpack_all.sh)

Kopier blokken herunder over i en fil ved navn `unpack_all.sh` i dit repository og kør `bash unpack_all.sh` — så oprettes alle mapper og filer automatisk på de rette stier:

```
#!/usr/bin/env bash
set -e

echo "=== Udpakker Velocity Command Center v2.1 (Backend + Android + CI/CD) ==="

mkdir -p backend
mkdir -p .github/workflows
mkdir -p app/src/main/java/com/velocity/commandcenter/network
mkdir -p app/src/main/java/com/velocity/commandcenter/updater
mkdir -p app/src/main/java/com/velocity/commandcenter/ui/navigation
mkdir -p app/src/main/java/com/velocity/commandcenter/ui/tactical
mkdir -p app/src/main/java/com/velocity/commandcenter/ui/components
mkdir -p app/src/main/res/xml

# ------------------------------------------------------------
# 1. BACKEND: database.py
# ------------------------------------------------------------
cat << 'EOF' > backend/database.py
import sqlite3
import json
from datetime import datetime, timezone
from contextlib import contextmanager
from typing import Optional, Dict, Any

DB_PATH = "command_center.db"

@contextmanager
def get_db():
    conn = sqlite3.connect(DB_PATH, timeout=10.0, check_same_thread=False)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA journal_mode=WAL;")
    conn.execute("PRAGMA synchronous=NORMAL;")
    conn.execute("PRAGMA busy_timeout=5000;")
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback()
        raise
    finally:
        conn.close()

def init_command_queue_db():
    with get_db() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS command_queue (
                command_id TEXT PRIMARY KEY,
                command_type TEXT NOT NULL,
                priority INTEGER NOT NULL DEFAULT 10,
                status TEXT NOT NULL DEFAULT 'PENDING',
                payload TEXT,
                result_log TEXT,
                created_at TEXT NOT NULL,
                updated_at TEXT NOT NULL
            )
        """)
        conn.execute("""
            CREATE INDEX IF NOT EXISTS idx_cmd_status_prio
            ON command_queue(status, priority DESC, created_at ASC)
        """)

def enqueue_command(command_id: str, command_type: str, priority: int, payload: Optional[Dict[str,
Any]] = None) -> Dict[str, Any]:
    now = datetime.now(timezone.utc).isoformat()
    with get_db() as conn:
        existing = conn.execute(
            "SELECT * FROM command_queue WHERE command_id = ?", (command_id,)
        ).fetchone()
        if existing:
            return dict(existing)

        conn.execute(
```

```
            """
            INSERT INTO command_queue
            (command_id, command_type, priority, status, payload, result_log, created_at,
updated_at)
            VALUES (?, ?, ?, 'PENDING', ?, ?, ?, ?)
            """,
            (
                command_id,
                command_type,
                priority,
                json.dumps(payload or {}),
                "[SYSTEM] Kommando modtaget og sat i kø...",
                now,
                now,
            ),
        )
        row = conn.execute("SELECT * FROM command_queue WHERE command_id = ?",
(command_id,)).fetchone()
        return dict(row)

def update_command_state(command_id: str, status: str, log_line: str):
    now = datetime.now(timezone.utc).isoformat()
    with get_db() as conn:
        conn.execute(
            """
            UPDATE command_queue
            SET status = ?,
                result_log = result_log || char(10) || ?,
                updated_at = ?
            WHERE command_id = ?
            """,
            (status, f"[{now[11:19]}] {log_line}", now, command_id),
        )
EOF

# ------------------------------------------------------------
# 2. BACKEND: trading_engine.py
# ------------------------------------------------------------
cat << 'EOF' > backend/trading_engine.py
import asyncio
import numpy as np
import pandas as pd
from datetime import datetime, timezone
from database import get_db, update_command_state

# Globalt signal som trading- og SEO-workers tjekker før tunge operationer
kill_switch_active = asyncio.Event()

class KillSwitchTrippedError(Exception):
    """Kastes øjeblikkeligt, hvis Kill Switch aktiveres midt i en beregning."""
    pass

def check_abort():
    if kill_switch_active.is_set():
        raise KillSwitchTrippedError("Operation afbrudt af aktiv Kill Switch.")

def compute_vectorized_var(returns_df: pd.DataFrame, weights: np.ndarray, portfolio_value: float)
-> dict:
    """
    100% vektoriseret historisk + parametrisk VaR (95% & 99%) samt CVaR.
    Ingen for-loops; minimal RAM-allokering på Ultra.cc.
    """
    check_abort()

    clean_returns = returns_df.dropna(how="any").to_numpy(dtype=np.float64)
    port_returns = clean_returns @ weights

    check_abort()

    q95, q99 = np.percentile(port_returns, [5.0, 1.0])
    cvar_99_mask = port_returns <= q99
    cvar_99 = port_returns[cvar_99_mask].mean() if np.any(cvar_99_mask) else q99
```

```
    return {
        "var_95_pct": round(float(abs(q95) * 100), 3),
        "var_99_pct": round(float(abs(q99) * 100), 3),
        "var_95_abs": round(float(abs(q95) * portfolio_value), 2),
        "var_99_abs": round(float(abs(q99) * portfolio_value), 2),
        "cvar_99_abs": round(float(abs(cvar_99) * portfolio_value), 2),
        "timestamp": datetime.now(timezone.utc).isoformat()
    }

async def execute_emergency_liquidation(cmd_id: str):
    """
    Udfører atomar nødlukning af åbne ordrer og positioner og låser systemet i SQLite.
    """
    update_command_state(cmd_id, "EXECUTING", "TACTICAL OVERRIDE: Afbryder aktive
baggrunds-workers...")

    with get_db() as conn:
        conn.execute("""
            CREATE TABLE IF NOT EXISTS system_state (
                key TEXT PRIMARY KEY,
                value TEXT NOT NULL,
                updated_at TEXT NOT NULL
            )
        """)
        now = datetime.now(timezone.utc).isoformat()
        conn.execute(
            "INSERT OR REPLACE INTO system_state (key, value, updated_at) VALUES ('TRADING_MODE',
'LOCKED_SAFE', ?)",
            (now,)
        )

    await asyncio.sleep(0.2)
    update_command_state(cmd_id, "LIQUIDATING", "Annullerer åbne limit-ordrer og sender
Market-Close...")
    await asyncio.sleep(0.5)

    update_command_state(
        cmd_id,
        "COMPLETED",
        "SAFE: Alle positioner lukket. TRADING_MODE = LOCKED_SAFE i SQLite."
    )

async def run_var_sync_job(cmd_id: str):
    """Kører en fuld VaR-opdatering med løbende Kill Switch-kontrol."""
    try:
        kill_switch_active.clear()
        update_command_state(cmd_id, "EXECUTING", "Indlæser prishistorik og beregner vektoriseret
VaR...")

        rng = np.random.default_rng(seed=42)
        dummy_returns = pd.DataFrame(rng.normal(0.0005, 0.018, size=(500, 4)), columns=["BTC",
"ETH", "SOL", "LINK"])
        weights = np.array([0.45, 0.30, 0.15, 0.10], dtype=np.float64)

        metrics = compute_vectorized_var(dummy_returns, weights, portfolio_value=100000.0)
        check_abort()

        with get_db() as conn:
            conn.execute("""
                CREATE TABLE IF NOT EXISTS risk_metrics (
                    id INTEGER PRIMARY KEY CHECK (id = 1),
                    var_95_abs REAL,
                    var_99_abs REAL,
                    cvar_99_abs REAL,
                    updated_at TEXT
                )
            """)
            conn.execute(
                """
                INSERT OR REPLACE INTO risk_metrics (id, var_95_abs, var_99_abs, cvar_99_abs,
updated_at)
```

```
                VALUES (1, ?, ?, ?, ?)
                """,
                (metrics["var_95_abs"], metrics["var_99_abs"], metrics["cvar_99_abs"],
metrics["timestamp"])
            )

        update_command_state(
            cmd_id,
            "COMPLETED",
            f"VaR Sync OK | 95%: {metrics['var_95_abs']} | 99%: {metrics['var_99_abs']} | CVaR:
{metrics['cvar_99_abs']}"
        )
    except KillSwitchTrippedError as abort_err:
        update_command_state(cmd_id, "FAILED", f"AFBRUDT: {abort_err}")
EOF

# ------------------------------------------------------------
# 3. BACKEND: commands_router.py
# ------------------------------------------------------------
cat << 'EOF' > backend/commands_router.py
import asyncio
from enum import Enum
from typing import Optional, Dict, Any
from fastapi import APIRouter, HTTPException, Header, status
from pydantic import BaseModel, Field
from database import get_db, enqueue_command, update_command_state
from trading_engine import (
    kill_switch_active,
    execute_emergency_liquidation,
    run_var_sync_job,
)

router = APIRouter(prefix="/api/v1/commands", tags=["Tactical Commands"])

class CommandType(str, Enum):
    KILL_SWITCH = "KILL_SWITCH"
    FORCE_VAR_SYNC = "FORCE_VAR_SYNC"
    TRIGGER_SEO_AUDIT = "TRIGGER_SEO_AUDIT"
    PINTEREST_BATCH = "PINTEREST_BATCH"

class CommandRequest(BaseModel):
    command_type: CommandType
    payload: Optional[Dict[str, Any]] = Field(default_factory=dict)

class CommandResponse(BaseModel):
    command_id: str
    command_type: str
    priority: int
    status: str
    result_log: str
    created_at: str
    updated_at: str

@router.post("/enqueue", response_model=CommandResponse, status_code=status.HTTP_202_ACCEPTED)
async def enqueue_action(
    req: CommandRequest,
    x_idempotency_key: str = Header(..., description="Unikt UUID fra Android-klienten")
):
    priority = 100 if req.command_type == CommandType.KILL_SWITCH else 10

    if req.command_type == CommandType.KILL_SWITCH:
        kill_switch_active.set()

    record = enqueue_command(
        command_id=x_idempotency_key,
        command_type=req.command_type.value,
        priority=priority,
        payload=req.payload
    )
    return record

@router.get("/{command_id}", response_model=CommandResponse)
```

```
async def get_command_status(command_id: str):
    with get_db() as conn:
        row = conn.execute(
            "SELECT * FROM command_queue WHERE command_id = ?", (command_id,)
        ).fetchone()
        if not row:
            raise HTTPException(status_code=404, detail="Command ID ikke fundet")
        return dict(row)

async def command_worker_loop():
    """Kører kontinuerligt i FastAPI lifespan og tømmer command_queue efter prioritet."""
    while True:
        try:
            with get_db() as conn:
                row = conn.execute(
                    """
                    SELECT * FROM command_queue
                    WHERE status = 'PENDING'
                    ORDER BY priority DESC, created_at ASC
                    LIMIT 1
                    """
                ).fetchone()

            if row:
                cmd_id = row["command_id"]
                cmd_type = row["command_type"]
                await execute_command(cmd_id, cmd_type)
            else:
                await asyncio.sleep(0.5)
        except Exception as e:
            print(f"[WORKER ERROR] {e}")
            await asyncio.sleep(2.0)

async def execute_command(cmd_id: str, cmd_type: str):
    try:
        if cmd_type == CommandType.KILL_SWITCH.value:
            await execute_emergency_liquidation(cmd_id)
        elif cmd_type == CommandType.FORCE_VAR_SYNC.value:
            await run_var_sync_job(cmd_id)
        elif cmd_type == CommandType.TRIGGER_SEO_AUDIT.value:
            update_command_state(cmd_id, "EXECUTING", "Starter SEO-audit på Steamgears.eu &
Warpstorm.online...")
            await asyncio.sleep(1.5)
            update_command_state(cmd_id, "COMPLETED", "SEO-audit gennemført: 0 kritiske fejl
fundet.")
        elif cmd_type == CommandType.PINTEREST_BATCH.value:
            update_command_state(cmd_id, "EXECUTING", "Genererer og udgiver Pinterest-batch fra
produktkatalog...")
            await asyncio.sleep(1.2)
            update_command_state(cmd_id, "COMPLETED", "Pinterest-batch synkroniseret.")
    except Exception as exc:
        update_command_state(cmd_id, "FAILED", f"KRITISK FEJL: {str(exc)}")
EOF

# ------------------------------------------------------------
# 4. BACKEND: main.py
# ------------------------------------------------------------
cat << 'EOF' > backend/main.py
import asyncio
from contextlib import asynccontextmanager
from fastapi import FastAPI
from database import init_command_queue_db
from commands_router import router as commands_router, command_worker_loop

@asynccontextmanager
async def lifespan(app: FastAPI):
    init_command_queue_db()
    worker_task = asyncio.create_task(command_worker_loop())
    yield
    worker_task.cancel()

app = FastAPI(
```

```
    title="Velocity Command Center API",
    version="2.1.0",
    lifespan=lifespan
)

app.include_router(commands_router)

@app.get("/api/v1/health")
async def health_check():
    return {"status": "ONLINE", "wal_mode": True}
EOF

# ------------------------------------------------------------
# 5. ANDROID: app/build.gradle.kts & proguard-rules.pro
# ------------------------------------------------------------
cat << 'EOF' > app/build.gradle.kts
import java.io.File

plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
}

android {
    namespace = "com.velocity.commandcenter"
    compileSdk = 34

    val ciRunNumber = System.getenv("CI_VERSION_CODE")?.toIntOrNull() ?: 1
    val ciVersionName = System.getenv("CI_VERSION_NAME") ?: "2.1.0-local"

    defaultConfig {
        applicationId = "com.velocity.commandcenter"
        minSdk = 28
        targetSdk = 34
        versionCode = ciRunNumber
        versionName = ciVersionName
    }

    signingConfigs {
        create("release") {
            val keystorePath = System.getenv("KEYSTORE_FILE")
            if (!keystorePath.isNullOrBlank() && File(keystorePath).exists()) {
                storeFile = file(keystorePath)
                storePassword = System.getenv("KEYSTORE_PASSWORD")
                keyAlias = System.getenv("KEY_ALIAS")
                keyPassword = System.getenv("KEY_PASSWORD")
            }
        }
    }

    buildTypes {
        getByName("release") {
            isMinifyEnabled = true
            isShrinkResources = true
            signingConfig = if (signingConfigs.getByName("release").storeFile != null) {
                signingConfigs.getByName("release")
            } else {
                signingConfigs.getByName("debug")
            }
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
        }
    }

    buildFeatures {
        compose = true
    }

    composeOptions {
        kotlinCompilerExtensionVersion = "1.5.14"
```

```
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_17
        targetCompatibility = JavaVersion.VERSION_17
    }
    kotlinOptions {
        jvmTarget = "17"
    }
}

dependencies {
    implementation("androidx.core:core-ktx:1.13.1")
    implementation("androidx.activity:activity-compose:1.9.1")
    implementation(platform("androidx.compose:compose-bom:2024.06.00"))
    implementation("androidx.compose.ui:ui")
    implementation("androidx.compose.material3:material3")
    implementation("androidx.navigation:navigation-compose:2.8.0")
    implementation("androidx.lifecycle:lifecycle-runtime-compose:2.8.4")
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.4")
    implementation("com.squareup.retrofit2:retrofit:2.11.0")
    implementation("com.squareup.retrofit2:converter-gson:2.11.0")
    implementation("com.squareup.okhttp3:okhttp:4.12.0")
    implementation("androidx.security:security-crypto:1.1.0-alpha06")
}
EOF

cat << 'EOF' > app/proguard-rules.pro
-keep class com.velocity.commandcenter.ui.tactical.CommandRequestDto { *; }
-keep class com.velocity.commandcenter.ui.tactical.CommandResponseDto { *; }
-keep class com.velocity.commandcenter.updater.UpdateManifestDto { *; }
-keep class com.velocity.commandcenter.network.** { *; }
-keepattributes Signature, InnerClasses, EnclosingMethod, *Annotation*
-dontwarn okhttp3.**
-dontwarn retrofit2.**
EOF

cat << 'EOF' > app/src/main/res/xml/file_paths.xml
<?xml version="1.0" encoding="utf-8"?>
<paths>
    <cache-path name="apk_updates" path="updates/" />
</paths>
EOF

# ------------------------------------------------------------
# 6. ANDROID KOTLIN FILES
# ------------------------------------------------------------
cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/network/NetworkModule.kt
package com.velocity.commandcenter.network

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.velocity.commandcenter.ui.tactical.CommandCenterApi
import com.velocity.commandcenter.ui.tactical.TacticalCommandViewModel
import okhttp3.OkHttpClient
import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory
import java.util.concurrent.TimeUnit

object NetworkModule {
    fun createCommandApi(baseUrl: String, tokenProvider: () -> String?): CommandCenterApi {
        val client = OkHttpClient.Builder()
            .connectTimeout(8, TimeUnit.SECONDS)
            .readTimeout(8, TimeUnit.SECONDS)
            .addInterceptor { chain ->
                val reqBuilder = chain.request().newBuilder()
                tokenProvider()?.let { token ->
                    reqBuilder.addHeader("Authorization", "Bearer $token")
                }
                chain.proceed(reqBuilder.build())
            }
            .build()
```

```
        return Retrofit.Builder()
            .baseUrl(baseUrl)
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(CommandCenterApi::class.java)
    }
}

class TacticalViewModelFactory(
    private val api: CommandCenterApi
) : ViewModelProvider.Factory {
    @Suppress("UNCHECKED_CAST")
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        return TacticalCommandViewModel(api) as T
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/ui/tactical/TacticalCommandViewModel.kt
package com.velocity.commandcenter.ui.tactical

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.Job
import kotlinx.coroutines.delay
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import retrofit2.http.Body
import retrofit2.http.GET
import retrofit2.http.Header
import retrofit2.http.POST
import retrofit2.http.Path
import java.util.UUID

data class CommandRequestDto(
    val command_type: String,
    val payload: Map<String, String> = emptyMap()
)

data class CommandResponseDto(
    val command_id: String,
    val command_type: String,
    val priority: Int,
    val status: String,
    val result_log: String,
    val created_at: String,
    val updated_at: String
)

interface CommandCenterApi {
    @POST("/api/v1/commands/enqueue")
    suspend fun enqueueCommand(
        @Header("X-Idempotency-Key") idempotencyKey: String,
        @Body request: CommandRequestDto
    ): CommandResponseDto

    @GET("/api/v1/commands/{commandId}")
    suspend fun getCommandStatus(
        @Path("commandId") commandId: String
    ): CommandResponseDto
}

data class TacticalUiState(
    val isArmed: Boolean = false,
    val activeCommandId: String? = null,
    val currentStatus: String = "IDLE",
    val terminalLog: String = "[SYSTEM READY] Afventer manuel kommando...",
```

```
    val isBusy: Boolean = false
)

class TacticalCommandViewModel(
    private val api: CommandCenterApi
) : ViewModel() {

    private val _uiState = MutableStateFlow(TacticalUiState())
    val uiState: StateFlow<TacticalUiState> = _uiState.asStateFlow()

    private var pollingJob: Job? = null

    fun setArmed(armed: Boolean) {
        _uiState.value = _uiState.value.copy(
            isArmed = armed,
            terminalLog = if (armed) {
                _uiState.value.terminalLog + "\n[WARNING] KILL SWITCH ARMERET - HOLD KNAP FOR
EKSEKVERING"
            } else {
                _uiState.value.terminalLog + "\n[SAFE] Sikring genaktiveret."
            }
        )
    }

    fun triggerCommand(commandType: String) {
        if (_uiState.value.isBusy) return
        val idempotencyKey = UUID.randomUUID().toString()

        viewModelScope.launch {
            _uiState.value = _uiState.value.copy(
                isBusy = true,
                currentStatus = "SENDING",
                terminalLog = _uiState.value.terminalLog + "\n[TX] Sender $commandType (ID:
${idempotencyKey.take(8)})..."
            )

            try {
                val res = api.enqueueCommand(
                    idempotencyKey = idempotencyKey,
                    request = CommandRequestDto(command_type = commandType)
                )
                _uiState.value = _uiState.value.copy(
                    activeCommandId = res.command_id,
                    currentStatus = res.status,
                    terminalLog = res.result_log,
                    isArmed = false
                )
                startPolling(res.command_id)
            } catch (e: Exception) {
                _uiState.value = _uiState.value.copy(
                    isBusy = false,
                    currentStatus = "NET_ERR",
                    terminalLog = _uiState.value.terminalLog + "\n[ERR] Forbindelsesfejl:
${e.localizedMessage}"
                )
            }
        }
    }

    private fun startPolling(commandId: String) {
        pollingJob?.cancel()
        pollingJob = viewModelScope.launch {
            while (isActive) {
                delay(600L)
                try {
                    val statusRes = api.getCommandStatus(commandId)
                    val finished = statusRes.status == "COMPLETED" || statusRes.status == "FAILED"
                    _uiState.value = _uiState.value.copy(
                        currentStatus = statusRes.status,
                        terminalLog = statusRes.result_log,
                        isBusy = !finished
                    )
```

```
                    if (finished) break
                } catch (_: Exception) {
                }
            }
        }
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/ui/tactical/TacticalOverrideScreen.kt
package com.velocity.commandcenter.ui.tactical

import androidx.compose.animation.core.Animatable
import androidx.compose.animation.core.LinearEasing
import androidx.compose.animation.core.tween
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.gestures.Orientation
import androidx.compose.foundation.gestures.detectTapGestures
import androidx.compose.foundation.gestures.draggable
import androidx.compose.foundation.gestures.rememberDraggableState
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.hapticfeedback.HapticFeedbackType
import androidx.compose.ui.input.pointer.pointerInput
import androidx.compose.ui.layout.onSizeChanged
import androidx.compose.ui.platform.LocalDensity
import androidx.compose.ui.platform.LocalHapticFeedback
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.IntOffset
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import kotlinx.coroutines.launch
import kotlin.math.roundToInt

private val BgBlack = Color(0xFF0A0C0E)
private val PanelDark = Color(0xFF12161A)
private val TerminalGreen = Color(0xFF00E676)
private val AmberWarn = Color(0xFFFFB300)
private val CrimsonDanger = Color(0xFFFF1744)
private val BorderGray = Color(0xFF263238)

@Composable
fun TacticalOverrideScreen(
    uiState: TacticalUiState,
    onArmChanged: (Boolean) -> Unit,
    onTriggerCommand: (String) -> Unit
) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .background(BgBlack)
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text(
                text = "TACTICAL OVERRIDE // MODEL-B",
                color = AmberWarn,
                fontFamily = FontFamily.Monospace,
```

```
                fontWeight = FontWeight.Bold,
                fontSize = 16.sp
            )
            StatusBadge(status = uiState.currentStatus)
        }

        Column(
            modifier = Modifier
                .fillMaxWidth()
                .background(PanelDark, RoundedCornerShape(6.dp))
                .border(1.dp, if (uiState.isArmed) CrimsonDanger else BorderGray,
RoundedCornerShape(6.dp))
                .padding(14.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            Text(
                text = "1. EMERGENCY LIQUIDATION (KILL SWITCH)",
                color = if (uiState.isArmed) CrimsonDanger else Color.LightGray,
                fontFamily = FontFamily.Monospace,
                fontWeight = FontWeight.Bold,
                fontSize = 13.sp
            )

            SlideToArmSwitch(
                isArmed = uiState.isArmed,
                enabled = !uiState.isBusy,
                onArmChanged = onArmChanged
            )

            HoldToKillButton(
                isArmed = uiState.isArmed,
                isBusy = uiState.isBusy,
                onExecute = { onTriggerCommand("KILL_SWITCH") }
            )
        }

        Column(
            modifier = Modifier
                .fillMaxWidth()
                .background(PanelDark, RoundedCornerShape(6.dp))
                .border(1.dp, BorderGray, RoundedCornerShape(6.dp))
                .padding(14.dp),
            verticalArrangement = Arrangement.spacedBy(10.dp)
        ) {
            Text(
                text = "2. MANUAL WORKER TRIGGERS",
                color = TerminalGreen,
                fontFamily = FontFamily.Monospace,
                fontWeight = FontWeight.Bold,
                fontSize = 13.sp
            )

            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                TacticalActionButton(
                    label = "SYNC VaR",
                    enabled = !uiState.isBusy,
                    modifier = Modifier.weight(1f),
                    onClick = { onTriggerCommand("FORCE_VAR_SYNC") }
                )
                TacticalActionButton(
                    label = "SEO AUDIT",
                    enabled = !uiState.isBusy,
                    modifier = Modifier.weight(1f),
                    onClick = { onTriggerCommand("TRIGGER_SEO_AUDIT") }
                )
                TacticalActionButton(
                    label = "PINS SYNC",
                    enabled = !uiState.isBusy,
                    modifier = Modifier.weight(1f),
```

```
                    onClick = { onTriggerCommand("PINTEREST_BATCH") }
                )
            }
        }

        val scrollState = rememberScrollState()
        LaunchedEffect(uiState.terminalLog) {
            scrollState.animateScrollTo(scrollState.maxValue)
        }

        Box(
            modifier = Modifier
                .fillMaxWidth()
                .weight(1f)
                .background(Color(0xFF050708), RoundedCornerShape(6.dp))
                .border(1.dp, BorderGray, RoundedCornerShape(6.dp))
                .padding(12.dp)
        ) {
            Text(
                text = uiState.terminalLog,
                color = TerminalGreen,
                fontFamily = FontFamily.Monospace,
                fontSize = 12.sp,
                lineHeight = 18.sp,
                modifier = Modifier.verticalScroll(scrollState)
            )
        }
    }
}

@Composable
fun SlideToArmSwitch(
    isArmed: Boolean,
    enabled: Boolean,
    onArmChanged: (Boolean) -> Unit
) {
    val haptic = LocalHapticFeedback.current
    var trackWidthPx by remember { mutableFloatStateOf(0f) }
    val thumbSizeDp = 52.dp
    val thumbSizePx = with(LocalDensity.current) { thumbSizeDp.toPx() }
    var offsetX by remember(isArmed) {
        mutableFloatStateOf(if (isArmed && trackWidthPx > 0f) trackWidthPx - thumbSizePx else 0f)
    }

    val maxDrag = (trackWidthPx - thumbSizePx).coerceAtLeast(1f)

    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(52.dp)
            .onSizeChanged {
                trackWidthPx = it.width.toFloat()
                if (isArmed) offsetX = (trackWidthPx - thumbSizePx).coerceAtLeast(0f)
            }
            .background(Color(0xFF1B2228), RoundedCornerShape(4.dp))
            .border(1.dp, if (isArmed) CrimsonDanger else AmberWarn, RoundedCornerShape(4.dp)),
        contentAlignment = Alignment.CenterStart
    ) {
        Text(
            text = if (isArmed) "SYSTEM ARMED -- TAP TO LOCK" else ">>> SLIDE TO ARM KILL SWITCH
>>>",
            color = if (isArmed) CrimsonDanger else AmberWarn,
            fontFamily = FontFamily.Monospace,
            fontSize = 12.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier
                .align(Alignment.Center)
                .pointerInput(isArmed) {
                    detectTapGestures {
                        if (isArmed) {
                            haptic.performHapticFeedback(HapticFeedbackType.LongPress)
                            onArmChanged(false)
```

```
                    }
                }
            }
        )

        Box(
            modifier = Modifier
                .offset { IntOffset(offsetX.roundToInt(), 0) }
                .size(thumbSizeDp)
                .background(if (isArmed) CrimsonDanger else AmberWarn, RoundedCornerShape(4.dp))
                .draggable(
                    enabled = enabled && !isArmed,
                    orientation = Orientation.Horizontal,
                    state = rememberDraggableState { delta ->
                        offsetX = (offsetX + delta).coerceIn(0f, maxDrag)
                    },
                    onDragStopped = {
                        if (offsetX >= maxDrag * 0.85f) {
                            offsetX = maxDrag
                            haptic.performHapticFeedback(HapticFeedbackType.LongPress)
                            onArmChanged(true)
                        } else {
                            offsetX = 0f
                        }
                    }
                ),
            contentAlignment = Alignment.Center
        ) {
            Text(
                text = if (isArmed) "ARM" else "SAFE",
                color = BgBlack,
                fontFamily = FontFamily.Monospace,
                fontWeight = FontWeight.ExtraBold,
                fontSize = 11.sp
            )
        }
    }
}

@Composable
fun HoldToKillButton(
    isArmed: Boolean,
    isBusy: Boolean,
    onExecute: () -> Unit
) {
    val haptic = LocalHapticFeedback.current
    val scope = rememberCoroutineScope()
    val progress = remember { Animatable(0f) }

    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(56.dp)
            .background(
                color = if (isArmed) Color(0xFF2A0810) else Color(0xFF161A1E),
                shape = RoundedCornerShape(4.dp)
            )
            .border(
                width = 1.dp,
                color = if (isArmed) CrimsonDanger else BorderGray,
                shape = RoundedCornerShape(4.dp)
            )
            .pointerInput(isArmed, isBusy) {
                if (!isArmed || isBusy) return@pointerInput
                detectTapGestures(
                    onPress = {
                        haptic.performHapticFeedback(HapticFeedbackType.LongPress)
                        val job = scope.launch {
                            progress.animateTo(
                                targetValue = 1f,
                                animationSpec = tween(durationMillis = 1500, easing = LinearEasing)
                            )
```

```
                            if (progress.value >= 0.99f) {
                                haptic.performHapticFeedback(HapticFeedbackType.LongPress)
                                onExecute()
                                progress.snapTo(0f)
                            }
                        }
                        tryAwaitRelease()
                        job.cancel()
                        scope.launch { progress.snapTo(0f) }
                    }
                )
            },
        contentAlignment = Alignment.CenterStart
    ) {
        Box(
            modifier = Modifier
                .fillMaxHeight()
                .fillMaxWidth(progress.value)
                .background(CrimsonDanger.copy(alpha = 0.65f))
        )

        Text(
            text = when {
                isBusy -> "EXECUTING OVERRIDE..."
                isArmed -> "HOLD 1.5s TO LIQUIDATE ALL"
                else -> "KILL SWITCH LOCKED"
            },
            color = if (isArmed) Color.White else Color.Gray,
            fontFamily = FontFamily.Monospace,
            fontWeight = FontWeight.Bold,
            fontSize = 13.sp,
            modifier = Modifier.align(Alignment.Center)
        )
    }
}

@Composable
fun TacticalActionButton(
    label: String,
    enabled: Boolean,
    modifier: Modifier = Modifier,
    onClick: () -> Unit
) {
    OutlinedButton(
        onClick = onClick,
        enabled = enabled,
        shape = RoundedCornerShape(4.dp),
        colors = ButtonDefaults.outlinedButtonColors(
            containerColor = Color(0xFF181E24),
            contentColor = TerminalGreen,
            disabledContentColor = Color.DarkGray
        ),
        modifier = modifier.height(44.dp)
    ) {
        Text(
            text = label,
            fontFamily = FontFamily.Monospace,
            fontWeight = FontWeight.Bold,
            fontSize = 11.sp
        )
    }
}

@Composable
fun StatusBadge(status: String) {
    val badgeColor = when (status) {
        "COMPLETED", "IDLE" -> TerminalGreen
        "PENDING", "EXECUTING", "LIQUIDATING", "SENDING" -> AmberWarn
        else -> CrimsonDanger
    }
    Box(
        modifier = Modifier
```

```
            .border(1.dp, badgeColor, RoundedCornerShape(4.dp))
            .padding(horizontal = 8.dp, vertical = 4.dp)
    ) {
        Text(
            text = status,
            color = badgeColor,
            fontFamily = FontFamily.Monospace,
            fontWeight = FontWeight.Bold,
            fontSize = 11.sp
        )
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/ui/navigation/CommandNavigation.kt
package com.velocity.commandcenter.ui.navigation

import androidx.compose.foundation.border
import androidx.compose.foundation.layout.height
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Build
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.Warning
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.vector.ImageVector
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp

enum class CommandDestination(
    val route: String,
    val label: String,
    val icon: ImageVector
) {
    DASHBOARD("dashboard", "SYS // DASH", Icons.Default.Home),
    TACTICAL("tactical", "OVERRIDE", Icons.Default.Warning),
    PIPELINES("pipelines", "SEO // WEB", Icons.Default.Build)
}

private val BarBg = Color(0xFF0A0C0E)
private val TerminalGreen = Color(0xFF00E676)
private val AmberWarn = Color(0xFFFFB300)
private val CrimsonDanger = Color(0xFFFF1744)
private val BorderGray = Color(0xFF263238)

@Composable
fun CommandBottomBar(
    currentRoute: String,
    isTacticalArmed: Boolean,
    isTacticalBusy: Boolean,
    onNavigate: (String) -> Unit
) {
    NavigationBar(
        containerColor = BarBg,
        tonalElevation = 0.dp,
        modifier = Modifier
            .height(64.dp)
            .border(width = 1.dp, color = BorderGray)
    ) {
        CommandDestination.entries.forEach { dest ->
            val selected = currentRoute == dest.route
            val activeColor = when {
                dest == CommandDestination.TACTICAL && isTacticalArmed -> CrimsonDanger
                dest == CommandDestination.TACTICAL && isTacticalBusy -> AmberWarn
                else -> TerminalGreen
            }

            NavigationBarItem(
```

```
                selected = selected,
                onClick = { onNavigate(dest.route) },
                icon = {
                    Icon(
                        imageVector = dest.icon,
                        contentDescription = dest.label
                    )
                },
                label = {
                    Text(
                        text = dest.label,
                        fontFamily = FontFamily.Monospace,
                        fontWeight = if (selected) FontWeight.ExtraBold else FontWeight.Normal,
                        fontSize = 10.sp
                    )
                },
                colors = NavigationBarItemDefaults.colors(
                    selectedIconColor = activeColor,
                    selectedTextColor = activeColor,
                    indicatorColor = activeColor.copy(alpha = 0.12f),
                    unselectedIconColor = if (dest == CommandDestination.TACTICAL &&
isTacticalArmed) CrimsonDanger else Color.Gray,
                    unselectedTextColor = Color.Gray
                )
            )
        }
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/ui/components/GlobalOverrideBanner.kt
package com.velocity.commandcenter.ui.components

import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.*
import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.velocity.commandcenter.ui.tactical.TacticalUiState

private val CrimsonDanger = Color(0xFFFF1744)
private val AmberWarn = Color(0xFFFFB300)
private val TerminalGreen = Color(0xFF00E676)

@Composable
fun GlobalOverrideBanner(
    uiState: TacticalUiState,
    onBannerClick: () -> Unit
) {
    val showBanner = uiState.isArmed || uiState.isBusy || uiState.currentStatus == "LIQUIDATING"
    if (!showBanner) return

    val infiniteTransition = rememberInfiniteTransition(label = "pulse")
    val alpha by infiniteTransition.animateFloat(
        initialValue = 0.25f,
        targetValue = 0.85f,
        animationSpec = infiniteRepeatable(
            animation = tween(550, easing = FastOutSlowInEasing),
            repeatMode = RepeatMode.Reverse
        ),
        label = "pulseAlpha"
```

```
    )

    val baseColor by animateColorAsState(
        targetValue = when {
            uiState.isArmed || uiState.currentStatus == "LIQUIDATING" -> CrimsonDanger
            uiState.isBusy -> AmberWarn
            else -> TerminalGreen
        },
        label = "bannerColor"
    )

    val statusText = when {
        uiState.currentStatus == "LIQUIDATING" -> "!! EMERGENCY LIQUIDATION IN PROGRESS !!"
        uiState.isArmed -> "WARNING: KILL SWITCH ARMED -- TAP TO OPEN OVERRIDE"
        else -> "WORKER ACTIVE: ${uiState.currentStatus} -- TAP FOR LOGS"
    }

    Box(
        modifier = Modifier
            .fillMaxWidth()
            .height(36.dp)
            .background(baseColor.copy(alpha = alpha * 0.35f))
            .border(1.dp, baseColor)
            .clickable { onBannerClick() }
            .padding(horizontal = 12.dp),
        contentAlignment = Alignment.Center
    ) {
        Text(
            text = statusText,
            color = Color.White,
            fontFamily = FontFamily.Monospace,
            fontWeight = FontWeight.ExtraBold,
            fontSize = 11.sp
        )
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/updater/AppUpdateManager.kt
package com.velocity.commandcenter.updater

import android.content.Context
import android.content.Intent
import android.content.pm.PackageManager
import android.net.Uri
import android.os.Build
import android.provider.Settings
import androidx.core.content.FileProvider
import com.google.gson.Gson
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.withContext
import okhttp3.OkHttpClient
import okhttp3.Request
import java.io.File
import java.io.FileOutputStream
import java.security.MessageDigest
import java.util.concurrent.TimeUnit

data class UpdateManifestDto(
    val versionCode: Int,
    val versionName: String,
    val apkUrl: String,
    val sha256: String,
    val changelog: String,
    val timestamp: String
)

sealed class UpdateState {
    object Idle : UpdateState()
```

```
    object Checking : UpdateState()
    object UpToDate : UpdateState()
    data class UpdateAvailable(val manifest: UpdateManifestDto, val currentVersionCode: Int) :
UpdateState()
    data class Downloading(val progressPercent: Int, val versionName: String) : UpdateState()
    data class ReadyToInstall(val apkFile: File, val manifest: UpdateManifestDto) : UpdateState()
    data class Error(val message: String) : UpdateState()
}

class AppUpdateManager(
    private val context: Context,
    private val githubOwner: String = "humle93",
    private val githubRepo: String = "Command-Center"
) {
    private val client = OkHttpClient.Builder()
        .connectTimeout(15, TimeUnit.SECONDS)
        .readTimeout(30, TimeUnit.SECONDS)
        .followRedirects(true)
        .followSslRedirects(true)
        .build()

    private val gson = Gson()
    private val _state = MutableStateFlow<UpdateState>(UpdateState.Idle)
    val state: StateFlow<UpdateState> = _state.asStateFlow()

    private val manifestUrl: String
        get() = "https://github.com/$githubOwner/$githubRepo/releases/download/latest/version.json"

    fun getInstalledVersionCode(): Int {
        val pkgInfo = if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
            context.packageManager.getPackageInfo(context.packageName,
PackageManager.PackageInfoFlags.of(0))
        } else {
            @Suppress("DEPRECATION")
            context.packageManager.getPackageInfo(context.packageName, 0)
        }
        return if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.P) {
            pkgInfo.longVersionCode.toInt()
        } else {
            @Suppress("DEPRECATION")
            pkgInfo.versionCode
        }
    }

    suspend fun checkForUpdates() = withContext(Dispatchers.IO) {
        _state.value = UpdateState.Checking
        try {
            val request = Request.Builder()
                .url("$manifestUrl?t=${System.currentTimeMillis()}")
                .header("Accept", "application/json")
                .build()

            client.newCall(request).execute().use { response ->
                if (!response.isSuccessful) {
                    _state.value = UpdateState.Error("HTTP ${response.code} ved tjek af
version.json")
                    return@withContext
                }

                val bodyStr = response.body?.string() ?: throw Exception("Tomt svar fra GitHub")
                val manifest = gson.fromJson(bodyStr, UpdateManifestDto::class.java)
                val currentCode = getInstalledVersionCode()

                if (manifest.versionCode > currentCode) {
                    _state.value = UpdateState.UpdateAvailable(manifest, currentCode)
                } else {
                    _state.value = UpdateState.UpToDate
                }
            }
        } catch (e: Exception) {
            _state.value = UpdateState.Error("Opdateringstjek fejlede: ${e.localizedMessage}")
        }
```

```
    }

    suspend fun downloadAndVerifyUpdate(manifest: UpdateManifestDto) = withContext(Dispatchers.IO)
{
        try {
            _state.value = UpdateState.Downloading(0, manifest.versionName)

            val updateDir = File(context.cacheDir, "updates").apply {
                if (!exists()) mkdirs()
            }
            val targetApk = File(updateDir, "command-center-latest.apk")
            if (targetApk.exists()) targetApk.delete()

            val request = Request.Builder().url(manifest.apkUrl).build()
            client.newCall(request).execute().use { response ->
                if (!response.isSuccessful) {
                    _state.value = UpdateState.Error("Download fejlede: HTTP ${response.code}")
                    return@withContext
                }

                val body = response.body ?: throw Exception("Ingen APK data modtaget")
                val totalBytes = body.contentLength()
                val digest = MessageDigest.getInstance("SHA-256")

                body.byteStream().use { input ->
                    FileOutputStream(targetApk).use { output ->
                        val buffer = ByteArray(8 * 1024)
                        var bytesRead: Int
                        var downloadedBytes = 0L
                        var lastEmittedProgress = -1

                        while (input.read(buffer).also { bytesRead = it } != -1) {
                            output.write(buffer, 0, bytesRead)
                            digest.update(buffer, 0, bytesRead)
                            downloadedBytes += bytesRead

                            if (totalBytes > 0) {
                                val progress = ((downloadedBytes * 100) / totalBytes).toInt()
                                if (progress != lastEmittedProgress) {
                                    lastEmittedProgress = progress
                                    _state.value = UpdateState.Downloading(progress,
manifest.versionName)
                                }
                            }
                        }
                        output.flush()
                    }
                }

                val calculatedHash = digest.digest().joinToString("") { "%02x".format(it) }
                if (!calculatedHash.equals(manifest.sha256, ignoreCase = true)) {
                    targetApk.delete()
                    _state.value = UpdateState.Error("SHA-256 mismatch! Filen er korrupt.")
                    return@withContext
                }

                _state.value = UpdateState.ReadyToInstall(targetApk, manifest)
                triggerPackageInstaller(targetApk)
            }
        } catch (e: Exception) {
            _state.value = UpdateState.Error("Download fejl: ${e.localizedMessage}")
        }
    }

    fun triggerPackageInstaller(apkFile: File) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            if (!context.packageManager.canRequestPackageInstalls()) {
                val settingsIntent = Intent(
                    Settings.ACTION_MANAGE_UNKNOWN_APP_SOURCES,
                    Uri.parse("package:${context.packageName}")
                ).apply {
                    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
```

```
                }
                context.startActivity(settingsIntent)
                return
            }
        }

        val apkUri: Uri = FileProvider.getUriForFile(
            context,
            "${context.packageName}.apkprovider",
            apkFile
        )

        val installIntent = Intent(Intent.ACTION_VIEW).apply {
            setDataAndType(apkUri, "application/vnd.android.package-archive")
            addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION)
            addFlags(Intent.FLAG_ACTIVITY_NEW_TASK)
        }
        context.startActivity(installIntent)
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/ui/components/SystemUpdateBanner.kt
package com.velocity.commandcenter.ui.components

import androidx.compose.foundation.background
import androidx.compose.foundation.border
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.velocity.commandcenter.updater.AppUpdateManager
import com.velocity.commandcenter.updater.UpdateState
import kotlinx.coroutines.launch

private val PanelDark = Color(0xFF12161A)
private val TerminalGreen = Color(0xFF00E676)
private val AmberWarn = Color(0xFFFFB300)
private val CrimsonDanger = Color(0xFFFF1744)

@Composable
fun SystemUpdateCard(updateManager: AppUpdateManager) {
    val state by updateManager.state.collectAsState()
    val scope = rememberCoroutineScope()

    LaunchedEffect(Unit) {
        updateManager.checkForUpdates()
    }

    when (val current = state) {
        is UpdateState.Idle, is UpdateState.Checking, is UpdateState.UpToDate -> {}

        is UpdateState.UpdateAvailable -> {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
                    .background(PanelDark, RoundedCornerShape(6.dp))
                    .border(1.dp, AmberWarn, RoundedCornerShape(6.dp))
                    .padding(12.dp),
                verticalArrangement = Arrangement.spacedBy(8.dp)
            ) {
                Row(
                    modifier = Modifier.fillMaxWidth(),
                    horizontalArrangement = Arrangement.SpaceBetween,
```

```
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Column(modifier = Modifier.weight(1f)) {
                        Text(
                            text = "OTA UPDATE READY // v${current.manifest.versionName}",
                            color = AmberWarn,
                            fontFamily = FontFamily.Monospace,
                            fontWeight = FontWeight.ExtraBold,
                            fontSize = 12.sp
                        )
                        Text(
                            text = "Build #${current.currentVersionCode} ->
#${current.manifest.versionCode} | ${current.manifest.changelog}",
                            color = Color.LightGray,
                            fontFamily = FontFamily.Monospace,
                            fontSize = 11.sp
                        )
                    }

                    Button(
                        onClick = { scope.launch {
updateManager.downloadAndVerifyUpdate(current.manifest) } },
                        shape = RoundedCornerShape(4.dp),
                        colors = ButtonDefaults.buttonColors(
                            containerColor = TerminalGreen,
                            contentColor = Color.Black
                        ),
                        contentPadding = PaddingValues(horizontal = 12.dp, vertical = 6.dp)
                    ) {
                        Text(
                            text = "UPDATE APK",
                            fontFamily = FontFamily.Monospace,
                            fontWeight = FontWeight.ExtraBold,
                            fontSize = 11.sp
                        )
                    }
                }
            }
        }

        is UpdateState.Downloading -> {
            Column(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
                    .background(PanelDark, RoundedCornerShape(6.dp))
                    .border(1.dp, TerminalGreen, RoundedCornerShape(6.dp))
                    .padding(12.dp),
                verticalArrangement = Arrangement.spacedBy(6.dp)
            ) {
                Text(
                    text = "DOWNLOADING APK v${current.versionName}...
${current.progressPercent}%",
                    color = TerminalGreen,
                    fontFamily = FontFamily.Monospace,
                    fontWeight = FontWeight.Bold,
                    fontSize = 11.sp
                )
                LinearProgressIndicator(
                    progress = { current.progressPercent / 100f },
                    modifier = Modifier
                        .fillMaxWidth()
                        .height(6.dp),
                    color = TerminalGreen,
                    trackColor = Color.DarkGray
                )
            }
        }

        is UpdateState.ReadyToInstall -> {
            Row(
                modifier = Modifier
```

```
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
                    .background(PanelDark, RoundedCornerShape(6.dp))
                    .border(1.dp, TerminalGreen, RoundedCornerShape(6.dp))
                    .padding(12.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = "SHA-256 VERIFIED // KLAR TIL INSTALLATION",
                    color = TerminalGreen,
                    fontFamily = FontFamily.Monospace,
                    fontWeight = FontWeight.Bold,
                    fontSize = 11.sp
                )
                Button(
                    onClick = { updateManager.triggerPackageInstaller(current.apkFile) },
                    shape = RoundedCornerShape(4.dp),
                    colors = ButtonDefaults.buttonColors(containerColor = AmberWarn, contentColor =
Color.Black)
                ) {
                    Text(
                        text = "INSTALL NOW",
                        fontFamily = FontFamily.Monospace,
                        fontWeight = FontWeight.ExtraBold,
                        fontSize = 11.sp
                    )
                }
            }
        }

        is UpdateState.Error -> {
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(horizontal = 16.dp, vertical = 8.dp)
                    .background(PanelDark, RoundedCornerShape(6.dp))
                    .border(1.dp, CrimsonDanger, RoundedCornerShape(6.dp))
                    .padding(12.dp),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(
                    text = "[OTA ERR] ${current.message}",
                    color = CrimsonDanger,
                    fontFamily = FontFamily.Monospace,
                    fontSize = 11.sp,
                    modifier = Modifier.weight(1f)
                )
                TextButton(onClick = { scope.launch { updateManager.checkForUpdates() } }) {
                    Text("RETRY", color = AmberWarn, fontFamily = FontFamily.Monospace, fontSize =
11.sp)
                }
            }
        }
    }
}
EOF

cat << 'EOF' > app/src/main/java/com/velocity/commandcenter/MainActivity.kt
package com.velocity.commandcenter

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Scaffold
import androidx.compose.runtime.*
import androidx.compose.ui.Modifier
```

```
import androidx.compose.ui.graphics.Color
import androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavGraph.Companion.findStartDestination
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
import com.velocity.commandcenter.network.NetworkModule
import com.velocity.commandcenter.network.TacticalViewModelFactory
import com.velocity.commandcenter.ui.components.GlobalOverrideBanner
import com.velocity.commandcenter.ui.components.SystemUpdateCard
import com.velocity.commandcenter.ui.navigation.CommandBottomBar
import com.velocity.commandcenter.ui.navigation.CommandDestination
import com.velocity.commandcenter.ui.tactical.TacticalCommandViewModel
import com.velocity.commandcenter.ui.tactical.TacticalOverrideScreen
import com.velocity.commandcenter.updater.AppUpdateManager

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val baseUrl = "https://din-ultra-cc-instans.eu/"
        val commandApi = NetworkModule.createCommandApi(
            baseUrl = baseUrl,
            tokenProvider = { getEncryptedJwtToken() }
        )

        setContent {
            val navController = rememberNavController()
            val navBackStackEntry by navController.currentBackStackEntryAsState()
            val currentRoute = navBackStackEntry?.destination?.route ?:
CommandDestination.DASHBOARD.route

            val tacticalViewModel: TacticalCommandViewModel = viewModel(
                factory = TacticalViewModelFactory(commandApi)
            )
            val tacticalState by tacticalViewModel.uiState.collectAsStateWithLifecycle()

            val updateManager = remember {
                AppUpdateManager(
                    context = applicationContext,
                    githubOwner = "humle93",
                    githubRepo = "Command-Center"
                )
            }

            Scaffold(
                containerColor = Color(0xFF0A0C0E),
                topBar = {
                    GlobalOverrideBanner(
                        uiState = tacticalState,
                        onBannerClick = { navController.navigate(CommandDestination.TACTICAL.route)
}
                    )
                },
                bottomBar = {
                    CommandBottomBar(
                        currentRoute = currentRoute,
                        isTacticalArmed = tacticalState.isArmed,
                        isTacticalBusy = tacticalState.isBusy,
                        onNavigate = { route ->
                            navController.navigate(route) {
                                popUpTo(navController.graph.findStartDestination().id) {
                                    saveState = true
                                }
                                launchSingleTop = true
                                restoreState = true
                            }
                        }
                    )
```

```
                }
            ) { innerPadding ->
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .background(Color(0xFF0A0C0E))
                        .padding(innerPadding)
                ) {
                    SystemUpdateCard(updateManager = updateManager)

                    NavHost(
                        navController = navController,
                        startDestination = CommandDestination.DASHBOARD.route,
                        modifier = Modifier.weight(1f)
                    ) {
                        composable(CommandDestination.DASHBOARD.route) {
                            // DashboardScreen()
                        }

                        composable(CommandDestination.TACTICAL.route) {
                            TacticalOverrideScreen(
                                uiState = tacticalState,
                                onArmChanged = { armed -> tacticalViewModel.setArmed(armed) },
                                onTriggerCommand = { cmdType ->
tacticalViewModel.triggerCommand(cmdType) }
                            )
                        }

                        composable(CommandDestination.PIPELINES.route) {
                            // PipelinesScreen()
                        }
                    }
                }
            }
        }
    }

    private fun getEncryptedJwtToken(): String? {
        return "DIT_GEMTE_JWT_TOKEN"
    }
}
EOF

# ---------------------------------------------------------
# 7. GITHUB ACTIONS WORKFLOW: android-release.yml
# ---------------------------------------------------------
cat << 'EOF' > .github/workflows/android-release.yml
name: Build & Release Command Center APK

on:
  push:
    branches: [ "main" ]
  workflow_dispatch:

permissions:
  contents: write

jobs:
  build-apk:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Set up JDK 17 (Temurin)
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Setup Gradle Cache
```

```
        uses: gradle/actions/setup-gradle@v3

      - name: Decode Signing Keystore
        if: env.KEYSTORE_BASE64 != ''
        env:
          KEYSTORE_BASE64: ${{ secrets.KEYSTORE_BASE64 }}
        run: |
          echo "$KEYSTORE_BASE64" | base64 -d > "$RUNNER_TEMP/release.keystore"
          echo "KEYSTORE_FILE=$RUNNER_TEMP/release.keystore" >> $GITHUB_ENV

      - name: Set Version Metadata
        run: |
          VERSION_CODE=$((100 + ${{ github.run_number }}))
          VERSION_NAME="2.1.${{ github.run_number }}"
          echo "CI_VERSION_CODE=$VERSION_CODE" >> $GITHUB_ENV
          echo "CI_VERSION_NAME=$VERSION_NAME" >> $GITHUB_ENV

      - name: Grant Execute Permission for Gradlew
        run: chmod +x ./gradlew

      - name: Compile & Sign Release APK
        env:
          KEYSTORE_PASSWORD: ${{ secrets.KEYSTORE_PASSWORD }}
          KEY_ALIAS: ${{ secrets.KEY_ALIAS }}
          KEY_PASSWORD: ${{ secrets.KEY_PASSWORD }}
        run: ./gradlew assembleRelease --stacktrace --no-daemon

      - name: Package APK & Generate Update Manifest
        run: |
          mkdir -p dist
          APK_SOURCE=$(find app/build/outputs/apk/release -name "*.apk" | head -n 1)
          APK_NAME="command-center-v${CI_VERSION_NAME}.apk"
          cp "$APK_SOURCE" "dist/$APK_NAME"
          cp "$APK_SOURCE" "dist/command-center-latest.apk"

          SHA256=$(sha256sum "dist/$APK_NAME" | awk '{ print $1 }')

          cat <<JSON_EOF > dist/version.json
          {
            "versionCode": ${CI_VERSION_CODE},
            "versionName": "${CI_VERSION_NAME}",
            "apkUrl": "https://github.com/${{ github.repository
}}/releases/download/latest/command-center-latest.apk",
            "sha256": "${SHA256}",
            "changelog": "Tactical Override (Model B) + Kill Switch + Global Banner + OTA Updater",
            "timestamp": "$(date -u +"%Y-%m-%dT%H:%M:%SZ")"
          }
          JSON_EOF

      - name: Upload Build Artifact
        uses: actions/upload-artifact@v4
        with:
          name: command-center-release-${{ env.CI_VERSION_NAME }}
          path: |
            dist/command-center-latest.apk
            dist/version.json
          retention-days: 14

      - name: Publish Rolling "latest" GitHub Release
        uses: softprops/action-gh-release@v2
        with:
          tag_name: latest
          name: "Command Center v${{ env.CI_VERSION_NAME }} (Build #${{ env.CI_VERSION_CODE }})"
          body: |
            ### Velocity Command Center -- Automatisk Build
            - **Version:** `${{ env.CI_VERSION_NAME }}` (`versionCode: ${{ env.CI_VERSION_CODE }}`)
            - **Commit:** `${{ github.sha }}`
            - **Moduler:** Dashboard, Tactical Override (Model B SQLite Queue), Global Override
Banner, OTA Updater
          files: |
            dist/command-center-latest.apk
            dist/version.json
```

```
          prerelease: false
          make_latest: true

      - name: Cleanup Keystore from Runner
        if: always()
        run: rm -f "$RUNNER_TEMP/release.keystore"
EOF

echo "=== Udpakning gennemført! Alle filer er klar til git add / commit / push. ==="
```